GCC: The GNU Compiler Collection — A Practical Flag Reference

`gcc` is the driver that turns C source into an executable. You rarely call the individual tools (`cpp`, `cc1`, `as`, `ld`) directly — `gcc` orchestrates them for you. This guide is a **working cheat-sheet of the flags you actually use**, grouped by purpose, with a runnable example for each group in `examples/gcc/`.

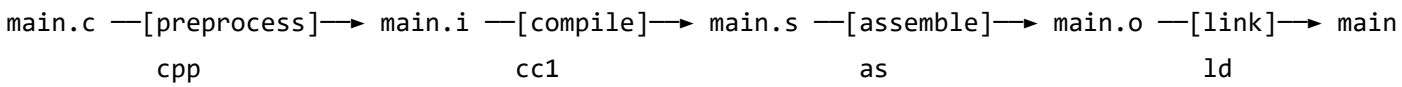
> Everything here applies to host `gcc` *and* to a cross compiler like `arm-none-eabi-gcc` — same flags, different target. See §12 Cross-compilation & embedded.

Contents

- GCC: The GNU Compiler Collection — A Practical Flag Reference
  - Contents
  - 1. The compilation pipeline
  - 2. Driving the stages (`-E`, `-S`, `-c`, `-o`)
  - 3. Warnings
  - 4. Language standard & dialect
  - 5. Optimization
  - 6. Debugging information
  - 7. Preprocessor: defines & includes
  - 8. Linking: libraries & paths
  - 9. Code-generation & size flags
  - 10. Diagnostics & analysis
  - 11. Sanitizers (host debugging)
  - 12. Cross-compilation & embedded
    - Selecting the CPU (ARM Cortex-M)
    - Building a bare-metal image
    - From ELF to a flashable image
    - Handy target-inspection flags
  - 13. Quick reference table
  - Further References

1. The compilation pipeline

A single `gcc main.c -o main` silently runs four stages:



Stage	What it does	Stop after it with
Preprocess	Expand `#include`, `#define`, `#if`	`-E`
Compile	C → assembly	`-S`
Assemble	Assembly → object code (`.o`)	`-c`
Link	Combine `.o` + libraries → executable	(default)

Understanding this makes almost every other flag obvious — each one targets a stage.
Demonstrated in examples/gcc/01-compilation-stages.

2. Driving the stages (`-E`, `-S`, `-c`, `-o`)

```
gcc -E main.c -o main.i     # preprocess only  → expanded source
gcc -S main.c -o main.s     # compile only     → assembly
gcc -c main.c -o main.o     # assemble only    → object file (no linking)
gcc main.o -o main          # link             → executable
gcc main.c -o main          # all four at once
```

- **`-o <file>`** names the output. Without it, executables default to `a.out`.
- **`-c`** is the one you use constantly in Makefiles: compile each `.c` to a `.o` independently, then link them all at the end. This is what enables incremental builds.
- **`-pipe`** passes data between stages via pipes instead of temp files (slightly faster).

3. Warnings

Warnings are your cheapest bug-finding tool. Turn them up.

Flag	Effect
`-Wall`	The "common mistakes" set (despite the name, *not* all warnings)
`-Wextra`	An extra batch on top of `-Wall`
`-Wpedantic`	Warn on anything not strictly standard-conforming
`-Werror`	Turn every warning into a hard error (great for CI)
`-Werror=<name>`	Make only *one* warning fatal, e.g. `-Werror=return-type`
`-Wshadow`	A local variable shadows another — classic bug source
`-Wconversion`	Implicit conversions that may change a value
`-Wcast-align`	Casts that increase required alignment (matters on ARM)
`-Wno-<name>`	Disable a specific warning, e.g. `-Wno-unused-parameter`

Recommended baseline for new code:

```
gcc -Wall -Wextra -Wpedantic -Wshadow -Wconversion main.c -o main
```

For embedded, add `-Wcast-align -Wpointer-arith -Wdouble-promotion` (the last catches accidental `double` math that pulls in slow soft-float routines).
See examples/gcc/02-warnings-standards.

4. Language standard & dialect

```
gcc -std=c11   main.c      # ISO C11 + GNU extensions allowed to error under -pedantic
gcc -std=gnu11 main.c      # C11 + GNU extensions (the gcc default varies by version)
gcc -std=c99   main.c
gcc -ansi      main.c      # same as -std=c90
```

- **`-std=cXX`** selects a strict ISO mode; **`-std=gnuXX`** allows GNU extensions.
- Pair a strict standard with **`-pedantic`** (or `-pedantic-errors`) to enforce it.
- For C++, use `g++` and `-std=c++17` / `c++20`, etc.

5. Optimization

Flag	Meaning	Use when
`-O0`	No optimization (default)	Debugging — code maps 1:1 to source
`-O1`	Basic optimizations	
`-O2`	Full "safe" optimization	Typical release builds
`-O3`	`-O2` + aggressive inlining/vectorization	CPU-bound host code
`-Os`	Optimize for **size**	**Embedded** — fit in flash
`-Og`	Optimize but keep debugging sane	**Best for stepping through code**
`-Ofast`	`-O3` + unsafe math (`-ffast-math`)	Only when you accept IEEE violations
`-flto`	Link-Time Optimization across translation units	Squeeze size/speed in release

Rules of thumb:

- **Debug builds:** `-Og -g3` . `-O0` also works but `-Og` keeps the code readable while still producing useful line info.
- **Embedded release:** `-Os` (often with `-flto`) to minimize flash footprint.
- Optimization changes behavior of undefined-behavior bugs — a bug that "works" at `-O0` can break at `-O2` . That's the optimizer exposing your UB, not a compiler bug.

See examples/gcc/03-optimization-debug.

6. Debugging information

```
gcc -g   main.c -o main      # default debug info
gcc -g3  main.c -o main      # -g plus macro definitions (best for GDB)
gcc -ggdb3 main.c            # richest GDB-specific info
```

- `-g` embeds symbols/line numbers so GDB can map addresses to source.

- `-g3` additionally records `#define` macros so you can inspect them in GDB.
- Debug info does **not** slow the code or (usually) end up in flash — for embedded it lives in the `.elf` and is stripped from the `.bin` / `.hex`. Keep the `.elf` for debugging even when you flash the stripped image.
- Strip symbols from a finished binary with `strip` or `-s` at link time.

7. Preprocessor: defines & includes

Flag	Effect
`-Dname`	Define `name` as `1`
`-Dname=value`	Define with a value, e.g. `-DSTM32F407xx -DHSE_VALUE=8000000`
`-Uname`	Undefine a macro
`-Idir`	Add `dir` to the header search path (`#include "..."` and `<...>`)
`-isystem dir`	Like `-I` but treated as a *system* header (suppresses its warnings)
`-include file`	Force-include a header before the source (e.g. a global config)
`-M` / `-MM`	Emit `make` -style dependency rules instead of compiling
`-MMD`	Generate `.d` dependency files *while* compiling (used for auto-deps)

```
gcc -DDEBUG=1 -DUSE_HAL_DRIVER -Iinclude -Idrivers/inc -c main.c
```

`-MMD -MP` is the idiom behind automatic header dependency tracking in Makefiles.
See examples/gcc/04-preprocessor-linking.

8. Linking: libraries & paths

Flag	Effect
`-llib`	Link `liblib.a` / `liblib.so` , e.g. `-lm` for libm
`-Ldir`	Add `dir` to the library search path

Flag	Effect
`-static`	Link statically (no shared-lib dependency at runtime)
`-shared`	Produce a shared library (`.so`)
`-Wl,<opt>`	Pass `<opt>` straight to the linker `ld`
`-T script.ld`	Use a custom linker script (essential on embedded)
`-nostartfiles`	Don't link the standard startup files (crt0)
`-nostdlib`	Link neither startup files nor standard libraries
`-nodefaultlibs`	Skip default libs but keep startup files

Key `-Wl,` passthroughs you'll use:

```
-Wl,--gc-sections          # drop unused sections (pairs with -ffunction-sections)
-Wl,-Map=output.map        # write a memory map file
-Wl,--print-memory-usage   # print flash/RAM usage after link (with a linker script)
-Wl,--start-group ... -Wl,--end-group   # resolve circular lib dependencies
```

Link order matters for static libraries: a library must appear on the command line *after* the objects that use it. `gcc main.o -lm` works; `gcc -lm main.o` may not.

9. Code-generation & size flags

These matter most on embedded, where every byte and cycle counts.

Flag	Effect
`-ffunction-sections`	Put each function in its own section
`-fdata-sections`	Put each data object in its own section
(link) `-Wl,--gc-sections`	Then let the linker garbage-collect unused sections
`-fno-common`	Put uninitialized globals in `.bss` , not COMMON (now the default)
`-fomit-frame-pointer`	Free up a register (implied by `-O` levels)

Flag	Effect
`-fno-exceptions -fno-rtti`	(C++) drop exception/RTTI overhead on MCUs
`-fstack-usage`	Emit a `.su` file with each function's stack usage
`-fstack-protector-strong`	Add stack-smashing guards (host security)
`-fPIC`	Position-independent code (needed for shared libraries)
`-fno-builtin`	Don't assume libc semantics for `memcpy`, `printf`, etc.

The trio **`-ffunction-sections -fdata-sections + -Wl,--gc-sections`** is the single most effective way to shrink an embedded image: unused functions and tables are dropped entirely. Check the `.map` file to see what survived.

10. Diagnostics & analysis

Flag	Effect
`-fanalyzer`	Static analyzer: detects leaks, double-free, NULL derefs (GCC 10+)
`-fdiagnostics-color=always`	Colorized errors
`-fmax-errors=N`	Stop after N errors
`-save-temps`	Keep the `.i`, `.s`, `.o` intermediates for inspection
`-v`	Verbose: show every subprocess and its flags
`-###`	Like `-v` but only *print* the commands, don't run them
`-print-search-dirs`	Show where gcc looks for programs/libraries
`-dM -E`	Dump all predefined macros (`gcc -dM -E - < /dev/null`)

`-v` and `-###` are invaluable when a cross build "can't find" a header or library — they show the exact search paths the driver is using.

11. Sanitizers (host debugging)

Runtime bug detectors. **Host only** — they need OS support, so not for bare metal, but perfect for testing your portable logic before it goes on-target.

```
gcc -fsanitize=address  -g main.c -o main   # ASan: use-after-free, overflows
gcc -fsanitize=undefined -g main.c -o main  # UBSan: signed overflow, bad shifts, UB
gcc -fsanitize=address,undefined -g main.c  # combine them
```

Run the resulting binary and it aborts with a precise report (file, line, stack) at the moment of the error. Catching UB on the host with UBSan is far cheaper than chasing a mysterious hard-fault on the MCU later.

12. Cross-compilation & embedded

Cross-compiling means the **build machine** (host) differs from the **target** that runs the code. You use a target-prefixed toolchain; the flags are the same `gcc` flags plus architecture selectors.

```
arm-none-eabi-gcc  ...   # ARM Cortex-M / Cortex-R / Cortex-A bare metal
riscv64-unknown-elf-gcc ...
avr-gcc ...
```

Selecting the CPU (ARM Cortex-M)

Flag	Meaning
`-mcpu=cortex-m4`	Target core (m0/m0plus/m3/m4/m7/m33…)
`-mthumb`	Use the Thumb instruction set (all Cortex-M is Thumb-only)
`-mfpu=fpv4-sp-d16`	FPU type (Cortex-M4F single-precision)
`-mfloat-abi=hard`	Pass floats in FPU registers (`soft` / `softfp` / `hard`)
`-mfloat-abi=soft`	No FPU; float math in software (M0/M0+/M3)

These CPU flags must be identical on every compile *and* on the link line, or objects won't link (mismatched float ABI). Typical Cortex-M4F set:

```
CPU="-mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard"
```

Building a bare-metal image

```
arm-none-eabi-gcc $CPU -Os -g3 \
    -ffunction-sections -fdata-sections \
    -Wall -Wextra \
    -c src/main.c -o build/main.o

arm-none-eabi-gcc $CPU \
    -T linker/cortex-m4.ld \
    -nostartfiles \
    -Wl,--gc-sections -Wl,-Map=build/fw.map \
    --specs=nano.specs --specs=nosys.specs \
    build/main.o build/startup.o -o build/fw.elf
```

Key embedded-specific link flags:

- **`-T linker/cortex-m4.ld`** — your linker script defines FLASH/RAM layout. Without it the default host layout is used and the image won't run on the chip.
- **`-nostartfiles`** — you provide the vector table and reset handler yourself.
- **`--specs=nano.specs`** — link against **newlib-nano**, a tiny libc for MCUs.
- **`--specs=nosys.specs`** — stub out OS syscalls (`_write` , `_sbrk` …) that a bare-metal system has no kernel for. (Alternatively `--specs=rdimon.specs` for semihosting.)

From ELF to a flashable image

```
arm-none-eabi-objcopy -O ihex   build/fw.elf build/fw.hex
arm-none-eabi-objcopy -O binary build/fw.elf build/fw.bin
arm-none-eabi-size build/fw.elf     # text/data/bss breakdown
```

- `.elf` keeps debug info — use it with GDB/OpenOCD.
- `.hex` / `.bin` are the stripped images you actually flash.

A full, buildable version of this lives in examples/gcc/05-cross-embedded and in the

CMake port at examples/cmake/03-cross-cortex-m4.

Handy target-inspection flags

```
arm-none-eabi-gcc $CPU -Q --help=target   # show every target flag's current value
arm-none-eabi-gcc -print-multi-lib        # list available multilib float/CPU variants
```

13. Quick reference table

Category	Flags you'll reach for daily
Stages	`-E` `-S` `-c` `-o` `-pipe` `-save-temps`
Warnings	`-Wall` `-Wextra` `-Wpedantic` `-Werror` `-Wshadow` `-Wconversion`
Standard	`-std=c11` `-std=gnu11` `-pedantic`
Optimize	`-O0` `-O2` `-Os` `-Og` `-O3` `-flto`
Debug	`-g` `-g3` `-ggdb3`
Preprocessor	`-Dx=y` `-Idir` `-isystem` `-include` `-MMD -MP`
Link	`-llib` `-Ldir` `-static` `-Wl,--gc-sections` `-Wl,-Map=f.map` `-T ldscript`
Codegen/size	`-ffunction-sections` `-fdata-sections` `-fstack-usage`
Analysis	`-fanalyzer` `-v` `-###` `-dM -E`
Sanitizers	`-fsanitize=address` `-fsanitize=undefined`
ARM target	`-mcpu=` `-mthumb` `-mfpu=` `-mfloat-abi=` `--specs=nano.specs`

Further References

- GCC manual: https://gcc.gnu.org/onlinedocs/gcc/
- "Options" chapters: *Invoking GCC*, *Optimize Options*, *Warning Options*.
- GNU Arm Embedded toolchain: https://developer.arm.com/downloads/-/gnu-rm

- `man gcc`, `man ld`, `man objcopy`, `info gcc`.